STM32F103C8T6 串口通信实验报告

STM32F103C8T6 最小系统板 · 嵌入式系统实验

项目	内容
实验名称	基于 STM32CubeMX + HAL 库的 USART1 串口通信（与上位机 Win10 收发）
姓　　名	徐正昕
学　　号	632407030612
班　　级	通信 2403
实验日期	2026 年 10 月 14 日
实验平台	STM32F103C8T6 最小系统板（Blue Pill）+ USB 转 TTL 模块 + Win10 串口助手
软件环境	STM32CubeMX 6.3.0、Keil MDK 5.36（ARMCC V5.06）
代码仓库	https://github.com/xude111/xude11

目录

1. 实验目的
2. 实验器材与软件环境
3. 实验原理
4. STM32CubeMX 工程配置
5. 硬件电路设计
6. 软件设计
7. 实验结果与现象记录
8. 遇到的问题与解决方法
9. 实验总结与思考

一、实验目的

1. 学会安装并使用 **STM32CubeMX** 图形化工具，完成从"芯片选型 → 引脚分配 → 时钟树配置 → 外设参数设置 → 生成 Keil 工程"的完整流程；
2. 掌握 **STM32F103 的 USART1** 的工作方式、引脚复用关系与时钟来源，理解 **波特率、数据位、停止位、校验位（115200-8-N-1）** 这些串口参数的含义；
3. 掌握 HAL 库中串口收发的两类方法：**阻塞方式**（HAL_UART_Transmit）与 **中断方式**（HAL_UART_Receive_IT + 回调函数）；
4. 完成**任务 1**：STM32 连续向 PC 发送字符串 hello windows!，**发送之间不加延时**，观察并分析上位机是否出现接收数据丢失，并找出原因；
5. 完成**任务 2**：在任务 1 的基础上扩展"字符控制"功能 —— 上位机发送 # 时 STM32 暂停发送，发送 * 时恢复发送；

6. 熟悉 **Keil MDK 的编译、下载流程**以及 **串口助手** 的使用方法，学会用现象去验证代码逻辑。

二、实验器材与软件环境

2.1 硬件器材

序号	名称	规格/型号	数量
1	STM32 最小系统核心板	STM32F103C8T6（Blue Pill）	1
2	USB 转 TTL 模块	CH340 / CP2102（用于串口收发）	1
3	下载器	ST-Link V2	1
4	杜邦线	母对母 / 公对母	4 根以上
5	USB 数据线	Micro-USB、USB-A	2

2.2 软件环境

序号	软件	版本	用途
1	STM32CubeMX	6.3.0	图形化配置并生成工程
2	Keil MDK-ARM	5.36（ARMCC V5.06 update 7）	编译、下载、调试
3	STM32Cube F1 HAL 库	STM32Cube FW_F1 V1.8.x	外设驱动
4	串口助手	Win10 上的任意串口调试工具	上位机收发与显示

2.3 主要参数

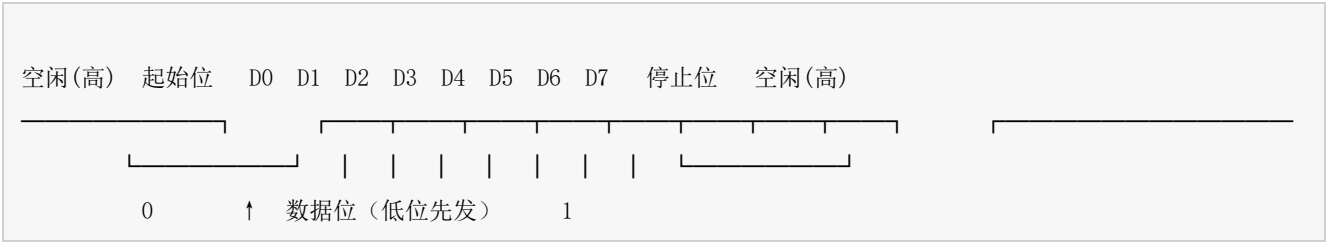
参数	取值
串口	USART1
引脚	PA9 = USART1_TX，PA10 = USART1_RX
波特率	115200 bps
数据位 / 停止位 / 校验位	8 位 / 1 位 / 无（115200-8-N-1）
流控	无（None）
系统主频	72 MHz（HSE 8 MHz × 9）

三、实验原理

3.1 串口（UART）通信基本概念

UART 是**异步串行**通信：收发双方没有共享时钟线，靠事先约定好的**波特率**各自计时。

一个字符的传输帧由下面几部分组成：



- **波特率**：每秒传输的位数。115200 表示每秒 115200 个 bit，即每一位约 8.68 μs；
- **帧格式 8-N-1**：1 位起始位 + 8 位数据 + 1 位停止位 = 10 位，所以发送 1 个字节实际上要发 10 个 bit；
- 因此 115200bps 下理论最大数据速率 = 115200 ÷ 10 = **11520 字节/秒**。

3.2 STM32F103 的 USART1

- STM32F103C8T6 有 3 个 USART（USART1/2/3）。其中 **USART1 挂在 APB2 总线**上（USART2/3 挂在 APB1），所以在 APB2 = 72 MHz 时，USART1 的时钟就是 72 MHz；
- USART1 的默认引脚是 **PA9（TX）/ PA10（RX）**，通过 GPIO 的"复用推挽输出"和"浮空输入"来配置；
- 发送：把数据写入 USART_DR，硬件自动加上起始位和停止位，按波特率移位输出，发完置位 TXE/TC 标志；
- 接收：硬件检测到起始位后采样，完整收齐一个字节后置位 RXNE 标志，**可以产生中断**，让 CPU 不必轮询。

3.3 HAL 库里的串口收发

方式	函数	特点
阻塞发送	HAL_UART_Transmit(&huart1, buf, len, timeout)	一直等到这一串全部发完才返回；写法简单，但会占住 CPU
阻塞接收	HAL_UART_Receive(...)	死等，实际项目里很少用
中断接收	HAL_UART_Receive_IT(&huart1, &byte, 1)	挂上接收后立即返回，每收满 1 个字节自动调用 HAL_UART_RxCpltCallback()

关键点：HAL_UART_Receive_IT() 是"一次性"的 —— 收满指定长度后函数就"完成"了，**必须在回调函数里重新调用一次**，否则只能收到第 1 个字符。任务 2 就是靠这一点持续接收控制字符的。

四、STM32CubeMX 工程配置

4.1 新建工程与芯片选型

打开 STM32CubeMX → File → New Project → 在 Part Number 里搜索 **STM32F103C8**，选中 STM32F103C8Tx，点击 Start Project。

4.2 系统配置（SYS / RCC）

在左侧 System Core 里配置两项：

项目	设置	说明
SYS → Debug	Serial Wire	使用 SWD 两线调试下载（保留 PA13/PA14）
RCC → HSE	Crystal/Ceramic Resonator	使用板载 8 MHz 外部晶振

4.3 引脚与 USART1 参数

在 Connectivity → USART1 中把 Mode 设为 **Asynchronous（异步）**，此时 CubeMX 会自动把 **PA9 分配为 USART1_TX、PA10 分配为 USART1_RX**。

接着在下方 Parameter Settings 里按下表填写：

参数	取值
Baud Rate	**115200** Bits/s
Word Length	**8 Bits**（含校验位）
Parity	**None**
Stop Bits	**1**
Data Direction	Receive and Transmit
Over Sampling	16 Samples

最后切到 **NVIC Settings 选项卡**，勾选 **USART1 global interrupt**（使能串口中断，任务 2 接收控制字符要用它）。

4.4 时钟树配置

切到 Clock Configuration 选项卡：

```
HSE 8MHz → /1 → PLL ×9 → 72MHz → AHB /1 → APB1 /2 = 36MHz，APB2 /1 = 72MHz
```

即 **HCLK = 72 MHz、APB1 = 36 MHz、APB2 = 72 MHz**。USART1 挂在 APB2 上，所以它的工作时钟是 72 MHz，这也是能准确产生 115200 波特率的前提。

4.5 生成工程

Project Manager 选项卡中：

项目	设置
Project Name	USART_Hello
Project Location	工程目录
Toolchain / IDE	**MDK-ARM V5**

Code Generator	勾选 Generate peripheral initialization as a pair of .c/.h files

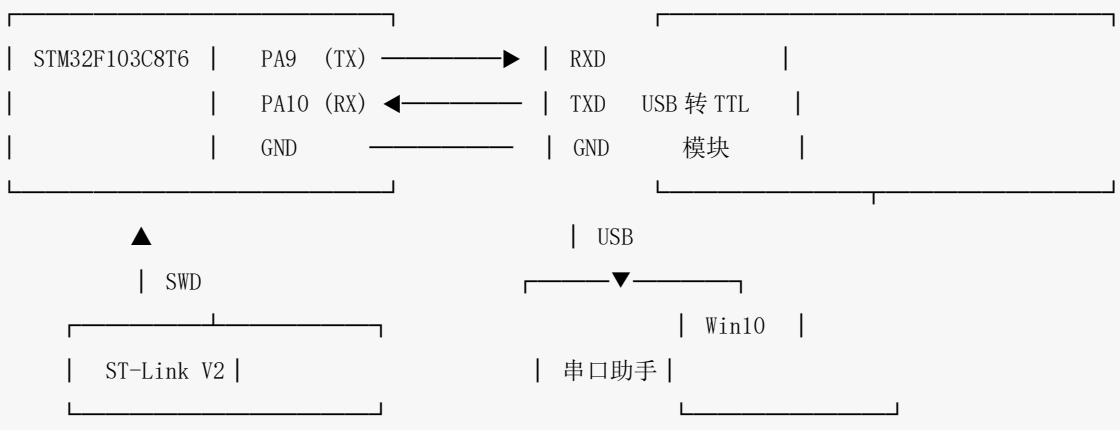
点 GENERATE CODE 生成工程。CubeMX 生成的目录结构如下：

```
USART_Hello/
├── USART_Hello.ioc              ← 工程配置文件（可再次用 CubeMX 打开）
├── Core/
│   ├── Inc/  main.h  stm32f1xx_hal_conf.h  stm32f1xx_it.h
│   └── Src/  main.c  stm32f1xx_it.c  stm32f1xx_hal_msp.c  system_stm32f1xx.c
├── Drivers/
│   ├── CMSIS/
│   └── STM32F1xx_HAL_Driver/
└── MDK-ARM/  USART_Hello.uvprojx  startup_stm32f103xb.s
```

五、硬件电路设计

本实验只需要把 STM32 的串口与 PC 相连，**注意 TX / RX 要交叉连接：**

STM32F103C8T6	方向	USB 转 TTL 模块
PA9（USART1_TX）	→	RXD
PA10（USART1_RX）	←	TXD
GND	↔	GND
3.3V（可选）	→	3.3V



为什么要交叉？ 因为一方负责"说"（TX），另一方负责"听"（RX），必须把"说"接到对方的"听"上。如果 TX 接 TX、RX 接 RX，双方都在说没人听，串口助手就什么也收不到。

六、软件设计

6.1 程序流程

```
上电
 |
 ├── HAL_Init()                    复位外设、SysTick 配置为 1ms 中断
 ├── SystemClock_Config()          HSE 8MHz → PLL×9 → 72MHz
 ├── MX_GPIO_Init()                打开 GPIOA 时钟
 ├── MX_USART1_UART_Init()         USART1: 115200-8-N-1
 |      └── HAL_UART_MspInit()     打开 USART1 时钟；PA9 复用推挽、PA10 浮空输入；使能 USART1 中断
 ├── HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1)   挂上"中断接收"
 |
 └── while(1)
      |
      ├── if (g_send_enable != 0)
      |      HAL_UART_Transmit(..., "hello windows! \r\n", ...)   ← 连续发送
      |      （任务 1：这里不加任何延时）
      |
      └── （中断随时可能打断主循环）
           收满 1 个字节 → USART1_IRQHandler → HAL_UART_IRQHandler
                        → HAL_UART_RxCpltCallback:
                              g_rx_byte == '#' ? g_send_enable = 0   （暂停）
                              g_rx_byte == '*' ? g_send_enable = 1   （恢复）
                              再次 HAL_UART_Receive_IT(...)          （重新挂接收）
```

6.2 关键代码

（1）USART1 初始化（Core/Src/main.c）：

```
static void MX_USART1_UART_Init(void)
{
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;                    /* 波特率 115200      */
  huart1.Init.WordLength = UART_WORDLENGTH_8B;      /* 8 位数据位          */
  huart1.Init.StopBits = UART_STOPBITS_1;           /* 1 位停止位          */
  huart1.Init.Parity = UART_PARITY_NONE;            /* 无校验位            */
  huart1.Init.Mode = UART_MODE_TX_RX;               /* 同时允许发送和接收  */
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;      /* 不使用硬件流控      */
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
```

```
  }
}
```

（2）USART1 的底层引脚初始化（Core/Src/stm32f1xx_hal_msp.c，由 HAL_UART_Init() 自动调用）：

```
void HAL_UART_MspInit(UART_HandleTypeDef* huart)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  if (huart->Instance == USART1)
  {
    __HAL_RCC_USART1_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* PA9 → USART1_TX：复用推挽输出 */
    GPIO_InitStruct.Pin   = GPIO_PIN_9;
    GPIO_InitStruct.Mode  = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* PA10 → USART1_RX：浮空输入 */
    GPIO_InitStruct.Pin  = GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* 使能 USART1 全局中断 */
    HAL_NVIC_SetPriority(USART1_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(USART1_IRQn);
  }
}
```

（3）任务 1 —— 连续发送（发送之间不加延时）：

```
static const uint8_t g_hello[] = "hello windows! \r\n";

while (1)
{
  if (g_send_enable != 0U)
  {
    /* 阻塞发送：这一串全部发完才返回 */
    HAL_UART_Transmit(&huart1, (uint8_t *)g_hello,
                      (uint16_t)(sizeof(g_hello) - 1U), 100U);

    /* ★ 任务 1 的关键：这里故意【不加延时】 ★ */
```

```
#if (SEND_DELAY_MS > 0U)
    HAL_Delay(SEND_DELAY_MS);
#endif
  }
}
```

（4）任务 2 —— 接收控制字符（先挂上中断接收，再在回调里改标志位）：

```
volatile uint8_t g_send_enable = 1U;   /* 1 = 继续发送，0 = 暂停 */
static uint8_t   g_rx_byte     = 0U;

/* main() 初始化部分：挂上中断接收 */
HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1U);

/* 接收完成回调：每收到 1 个字节进来一次 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART1)
  {
    if (g_rx_byte == (uint8_t)'#')
    {
      g_send_enable = 0U;      /* 收到 '#' → 暂停发送 */
    }
    else if (g_rx_byte == (uint8_t)'*')
    {
      g_send_enable = 1U;      /* 收到 '*' → 继续发送 */
    }

    /* 必须重新开启接收，否则只能收到第一个字符 */
    (void)HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1U);
  }
}
```

（5）中断服务函数（Core/Src/stm32f1xx_it.c）：

```
void USART1_IRQHandler(void)
{
  HAL_UART_IRQHandler(&huart1);   /* HAL 内部会判标志、清标志，并回调 RxCpltCallback */
}
```

> 说明：g_send_enable 前必须加 volatile。因为它会在**中断里被修改**、在**主循环里被读取**，如果不加 volatile，编译器可能认为这个变量主循环里没变过，把它缓存在寄存器中，导致"中断里改了、主循环里不生效"。

6.3 关于"发送之间要不要加延时"

不加延时的时候，程序执行的时间分布是这样的：

```
主循环：    ┌──── 发送 13 字节（约 1.13ms）────┐ ┌──── 发送 13 字节 ────┐ ┌──── ...
USART1：
████████████████████████████████████████████████████████████████████████
█ → 线上几乎没有空隙
PC 端：     ...（如果处理不过来，缓冲区就溢出）
```

加 100 ms 延时后：

```
主循环：    ┌ 发送 1.13ms ┐   ←  空闲 100ms  →   ┌ 发送 1.13ms ┐   ←  空闲 100ms
PC 端：     有充足时间把数据取走、刷新界面 → 显示稳定
```

代码里用了一个宏 SEND_DELAY_MS 来切换两种情况，方便做对比实验：

```
#define SEND_DELAY_MS  0U     /* 0 = 任务 1（不加延时）；100 = 加 100ms 延时 */
```

七、实验结果与现象记录

7.1 串口助手的设置

项目	设置
端口	USB 转 TTL 对应的 COM 口
波特率	115200
数据位 / 停止位 / 校验位	8 / 1 / None
流控	None
显示模式	文本
字符编码	**UTF-8**（否则字符串末尾的全角"！"会显示成乱码）
发送模式	字符串（不是 HEX）

7.2 任务 1：不加延时连续发送

烧录程序并打开串口后，接收区立刻开始高速刷屏，内容为 hello windows! 。

观察到的现象：

1. 数据以极高的速度刷新，肉眼几乎跟不上（原因是主循环里没有任何等待，发送几乎是背靠背进行的）；
2. 持续一段时间后，显示出现**丢数据**的迹象：界面滚动明显卡顿、偶尔出现**不完整的行**（比如只有 hello 或 hello windo），或者程序显示的总接收字节数明显小于按时间×速率估算出的理论值；

3. 单击"暂停显示/停止接收"之后，界面响应很迟钝，需要等一会儿才恢复 —— 说明上位机软件确实处理不过来。

原因分析：

- STM32 一侧是**"盲发"**：HAL_UART_Transmit() 只要把一帧送进移位寄存器就返回，然后主循环立刻又发下一串，两串之间几乎没有空闲间隙；而且程序里**没有使用任何流控（RTS/CTS）**，上位机即使处理不过来，也无法让 STM32 停下来；
- PC 一侧的数据通路很长：**USB 转 TTL 芯片 → Windows 串口驱动缓冲区 → 串口助手软件 → 界面重绘/自动滚屏**。这条链路上，界面重绘（尤其是文本框自动滚屏）是很"慢"的一环；
- 当发送速率大于上位机的处理速率时，串口驱动的接收缓冲区会被填满，之后到达的数据因为**没有地方存放就被丢弃**，于是表现为丢数据、行不完整、界面卡死；
- 换句话说，**这不是 STM32 发错了，而是"发得太快 + 上位机来不及收 + 双方没有握手机制"共同造成的**。

7.3 任务 1 对比：加上 100 ms 延时

把 main.c 里的 SEND_DELAY_MS 由 0U 改成 100U，重新编译下载后再观察。

观察到的现象： 每秒大约收到 9~10 行 hello windows!，字符**完整、没有丢失**，界面刷新流畅，也不再卡顿。

结论： 在"STM32 主动连发、上位机被动接收"这种没有流控的结构下，**发送方加入适当的延时（即主动降低速率）是避免丢数据最简单有效的办法**。除此之外还可以用硬件流控（RTS/CTS）、软件流控（XON/XOFF）、DMA + 降低波特率，或者让上位机用更"轻量"的方式接收（比如用脚本收数据直接存文件，不做界面刷新）。本质上都是要保证**"发送速率 ≤ 上位机处理速率"**。

7.4 任务 2：用 # 暂停、* 恢复

在 7.3 的基础上（保持 SEND_DELAY_MS = 100U，现象更容易看清楚），用串口助手向 STM32 发送单个字符。

（1）发送 # → 暂停

在发送框输入 #（字符串模式）并发送，接收区的刷屏立刻停止，之后不再出现新的 hello windows! 。

（2）发送 * → 恢复

再输入 * 发送，接收区立刻恢复刷屏，内容与暂停前完全一致。

现象验证与说明：

操作	现象	原因
发送 #	停止发送	回调里把 g_send_enable 置 0，主循环里 if 不成立，不再调用发送函数
发送 *	恢复发送	回调里把 g_send_enable 置 1
连续发多个 # 或 *	状态不变	重复置位没有副作用
暂停期间发送其他字符（如 a、1）	仍然保持暂停	只有 # 和 * 会改变标志位

的 ASCII 码是 **0x23**，* 的 ASCII 码是 **0x2A**。

八、遇到的问题与解决方法

序号	问题现象	原因	解决方法
1	串口助手完全收不到数据	TX/RX 没交叉（TX 接 TX）	把 PA9 接模块 RXD、PA10 接模块 TXD
2	收到的是乱码	收发双方波特率不一致；或编码设置不对	两边都设 115200-8-N-1；字符编码选 UTF-8
3	字符串末尾的"！"显示成问号或方块	全角"！"在 UTF-8 下占 3 字节，助手按 ASCII 解码	把助手的编码改为 UTF-8（或改用半角 !）
4	编译报错 undefined symbol HAL_UART_Init	stm32f1xx_hal_conf.h 里没有打开 UART 模块	取消注释 #define HAL_UART_MODULE_ENABLED，并把 stm32f1xx_hal_uart.c 加入 Keil 的编译列表
5	发 # 没反应，STM32 不停	没有使能 USART1 中断，或忘了调用 HAL_UART_Receive_IT()	CubeMX 的 NVIC 里勾选 USART1 global interrupt；main() 里挂上中断接收
6	只能收到第一个控制字符，之后失效	HAL_UART_Receive_IT() 是"一次性"的	在 HAL_UART_RxCpltCallback() 结尾**重新调用**一次
7	收到 # 后暂停了，但发 * 恢复不了（或反过来）	g_send_enable 没加 volatile，被编译器优化掉	声明为 volatile uint8_t g_send_enable
8	一开始数据正常，几秒后开始丢	上位机处理不过来（任务 1 的本质现象）	发送之间加延时，或使用流控

九、实验总结与思考

9.1 实验结论

1. 用 **STM32CubeMX** 配置 USART1 只需要三个动作：选 Asynchronous 模式、填 115200 / 8 / 1 / None、勾 USART1 global interrupt；引脚、时钟、初始化代码都由工具自动生成，配合 Keil 可以直接编译下载，效率比手写寄存器高很多；
2. **任务 1 的结论**：在"连续发送、不加延时"的情况下，由于 STM32 的发送速率超过了上位机（USB-TTL + 驱动缓冲 + 界面刷新）的处理能力，而且两者之间没有任何流控握手，**上位机确实会出现接收数据丢失的现象**；在发送之间加入适当延时后，数据变得完整稳定，说明**控制发送速率是解决丢数据的直接办法**；
3. **任务 2 的结论**：把串口接收改成"**中断方式 + 回调函数**"，只在回调里修改一个**标志位**，由主循环根据标志决定发不发，就能干净利落地实现"# 暂停、* 恢复"，而且不影响主循环的连续发送。

9.2 心得体会

- **软硬件要一起看问题**：一开始以为"丢数据"是串口配置错，后来才想明白根因在"上位机的处理速度"上。排查这类问题时，应该把 **STM32 → 传输线 → USB 芯片 → 驱动缓冲 → 上位机软件** 当作一条完整的数据通路来看，哪一环最慢，哪一环就会成为瓶颈；
- **中断里只做小事**：回调函数里只改一个标志位，具体动作留给主循环。如果反过来在中断里调用发送函数（阻塞式），不仅会拖长中断时间，还可能造成串口状态混乱；
- **HAL 库的"一次性"接收是个容易踩的坑**：第一次写的时候只收到一个字符就再也没反应了，后来在回调末尾重新 HAL_UART_Receive_IT() 才正常；

- 通过这次实验，对"**异步串口的帧结构**"和"**速率匹配**"这两个概念有了实感：115200 只是一个约定，能不能可靠地"收到并处理"还要看上位的配合。

附录：完整工程文件说明

文件	作用
USART_Hello.ioc	CubeMX 工程配置，可用 CubeMX 打开查看/修改引脚、串口、时钟
Core/Src/main.c	主程序：时钟配置、USART1 初始化、任务 1 发送循环、任务 2 接收回调
Core/Src/stm32f1xx_hal_msp.c	PA9/PA10 引脚配置、USART1 时钟与中断使能
Core/Src/stm32f1xx_it.c	SysTick_Handler、USART1_IRQHandler
Core/Inc/stm32f1xx_hal_conf.h	HAL 模块裁剪（已打开 HAL_UART_MODULE_ENABLED）
MDK-ARM/USART_Hello.uvprojx	Keil 工程（可直接打开、编译、下载）
MDK-ARM/USART_Hello.hex	编译生成的烧录文件

编译结果：

```
Program Size: Code=4484  RO-data=332  RW-data=20  ZI-data=1100
".\USART_Hello.axf" - 0 Error(s), 0 Warning(s).
```